

ui.select 全面详解（基于 NiceGUI 文档）

ui.select 是 NiceGUI 中用于实现单选项或多选项下拉选择的核心组件，基于 Quasar 的 QSelect 组件实现。它支持静态 / 动态选项配置、搜索过滤、多选模式、自定义选项内容等高级功能，具备数据绑定、状态监听、样式定制等能力，适用于表单录入、筛选条件、数据分类等需要从多个选项中选择场景。以下从核心特性、基础用法、高级功能等维度展开全面解析。

一、核心基础

1. 组件本质与核心特性

- 底层依赖：基于 Quasar 的 QSelect 组件，继承其成熟的下拉交互逻辑、搜索过滤机制、样式体系和无障碍支持。
- 核心功能：支持「单选」「多选」两种模式，选项可配置为列表（值即标签）或字典（键为值、值为标签），支持搜索、清空、禁用选项等功能。
- 灵活扩展性：通过 `ui.teleport` 可向选项中注入任意内容（如图标、图片、按钮），支持自定义下拉菜单样式和触发方式。

2. 初始化参数（核心配置）

参数名	类型	说明	默认值
options	list / dict / Callable	选项配置：- 列表：[值1, 值2, ...]（值即标签）- 字典：{值1: 标签1, 值2: 标签2, ...} - 可调用对象：返回列表 / 字典的函数（动态加载选项）	-
value	Any / list	初始选中值：- 单选模式：单个值（与选项值类型一致）- 多选模式：列表（ <code>multiple=True</code> 时）	None
on_change	Callable	选中值变化时触发的回调函数（ <code>e.value</code> 为当前选中值）	-
multiple	bool	是否启用多选模式（选中值为列表，支持按住 Ctrl 点击多选）	False
searchable	bool	是否启用搜索功能（下拉菜单顶部显示搜索框）	False
clearable	bool	是否支持清空选中值（显示清空按钮）	False
placeholder	str	未选中时的提示文本	""
disabled	bool	是否禁用组件（禁用后不可交互，视觉灰度）	False
label	str	组件标签（显示在选择框上方）	None
hint	str	提示文本（显示在选择框下方）	None

二、基础使用示例

1. 核心用法全覆盖（单选 / 多选 / 搜索 / 清空）

展示单选、多选、搜索 able、clearable、带标签 / 提示等基础用法，覆盖核心场景：

```
from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 单选模式（字典选项，值与标签分离）
    ui.select(
        options={'python': 'Python', 'js': 'JavaScript', 'java': 'Java'},
        label='编程语言',
        hint='选择你熟悉的编程语言',
        on_change=lambda e: ui.notify(f'选中: {e.value}')
    )

    # 2. 多选模式（列表选项，支持清空）
    ui.select(
        options=['红色', '绿色', '蓝色', '黄色'],
        multiple=True,
        clearable=True,
        label='喜欢的颜色',
        value=['红色', '蓝色'], # 初始选中值
        on_change=lambda e: ui.notify(f'选中: {e.value}')
    )

    # 3. 支持搜索的单选模式
    ui.select(
        options=['Apple', 'Banana', 'Cherry', 'Date', 'Grape', 'Mango', 'Orange'],
        searchable=True,
        placeholder='搜索水果...',
        label='水果选择',
        on_change=lambda e: ui.notify(f'选中: {e.value}')
    )

    # 4. 禁用状态的单选模式
    ui.select(
        options=['禁用选项1', '禁用选项2'],
        disabled=True,
        label='禁用的选择框',
        value='禁用选项1'
    )

ui.run()
```

效果：

- 四个选择框分别对应不同功能，支持标签、提示文本、搜索、清空、多选等特性；
- 多选模式下，按住 Ctrl 键可点击选择多个选项，点击清空按钮可取消所有选中；
- 搜索模式下，输入关键词可实时过滤选项，仅显示匹配结果。

2. 动态加载选项（可调用对象配置）

通过可调用对象配置 `options`，实现选项的动态加载（如从数据库、接口获取选项）：

```
from nicegui import ui

# 模拟从接口获取选项的函数
def get_dynamic_options():
    # 实际场景中可替换为数据库查询或 API 调用
    return {
        'user1': '张三',
        'user2': '李四',
        'user3': '王五'
    }

# 选项配置为可调用对象，组件初始化时自动调用加载选项
ui.select(
    options=get_dynamic_options,
    label='动态选项（用户列表）',
    on_change=lambda e: ui.notify(f'选中用户ID: {e.value}')
)

ui.run()
```

效果：组件初始化时自动调用 `get_dynamic_options` 函数加载选项，下拉菜单显示动态获取的用户列表。

三、核心功能与进阶用法

1. 多选模式高级配置（分隔符、最大选中数）

通过 `props` 配置多选模式的分隔符、最大选中数，优化多选体验：

```
from nicegui import ui

ui.select(
    options=['选项1', '选项2', '选项3', '选项4', '选项5'],
    multiple=True,
    clearable=True,
    label='多选配置示例',
    props='separator=, max-values=3', # separator: 选中值分隔符; max-values: 最大选中数
    hint='最多选择3个选项，选中值用逗号分隔',
    on_change=lambda e: ui.notify(f'选中: {e.value}')
)

ui.run()
```

效果：多选时最多可选择 3 个选项，超过则无法继续选择；输入框中选中值以逗号分隔显示。

2. 自定义选项内容（图标 + 文本）

通过 `ui.teleport` 向选项中注入图标，实现自定义选项样式，突破默认文本限制：

```
from nicegui import ui

# 选项配置（值为图标名称，标签为空）
options = ['home', 'settings', 'notifications', 'help']
select = ui.select(
    options={icon: '' for icon in options},
    label='图标选择',
    placeholder='选择一个图标',
    on_change=lambda e: ui.notify(f'选中图标: {e.value}')
)

# 向每个选项注入图标
for idx, icon in enumerate(options, 1):
    with ui.teleport(f'#{select.html_id} .q-select__options > div:nth-child({idx}) .q-item__label'):
        ui.icon(icon, color='blue-500').classes('mr-2')
        ui.label(icon.replace('_', ' ').capitalize()) # 图标名称转为文本标签

ui.run()
```

效果：下拉选项显示「图标 + 文本」组合，点击选项后，选择框中显示图标名称，实现自定义选项内容。

3. 数据绑定（双向同步）

通过 `bind_value` 实现 `select` 与数据对象的双向绑定，数据变化时组件自动更新，反之亦然：

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.selected_city = 'beijing' # 初始选中值
        self.selected_tags = ['tag1', 'tag2'] # 多选初始值

state = AppState()

with ui.column().classes('gap-4'):
    # 单选绑定
    ui.select(
        options={'beijing': '北京', 'shanghai': '上海', 'guangzhou': '广州'},
        label='城市选择',
    ).bind_value(state, 'selected_city')

    # 多选绑定
    ui.select(
        options={'tag1': '标签1', 'tag2': '标签2', 'tag3': '标签3', 'tag4': '标签4'},
        multiple=True,
        label='标签选择',
    ).bind_value(state, 'selected_tags')
```

```

# 显示绑定数据的实时状态
ui.label().bind_text_from(
    state, 'selected_city',
    lambda x: f'当前城市: {x}' ({state.selected_city}) '
)
ui.label().bind_text_from(
    state, 'selected_tags',
    lambda x: f'当前标签: {x}'
)

# 手动修改数据对象，组件自动同步
ui.button('切换为上海', on_click=lambda: setattr(state, 'selected_city',
'shanghai')).classes('mt-2')
ui.button('添加标签3', on_click=lambda: state.selected_tags.append('tag3')).classes('mt-2')

ui.run()

```

效果:

- 点击 select 切换选项，数据对象 `state` 的属性自动同步；
- 点击按钮修改数据对象，select 组件的选中状态自动更新；
- 下方标签实时显示数据对象的当前状态，实现双向绑定同步。

4. 样式定制（颜色、尺寸、下拉菜单样式）

通过 `props`、`classes`、`style` 定制 select 的颜色、尺寸、下拉菜单样式：

```

from nicegui import ui

with ui.column().classes('gap-4'):
    # 1. 自定义选中颜色和尺寸
    ui.select(
        options=['选项1', '选项2', '选项3'],
        label='自定义颜色',
        props='color=teal-500 size=lg', # color: 选中颜色; size: 组件尺寸 (lg=大)
    )

    # 2. 紧凑模式+圆角样式
    ui.select(
        options=['紧凑选项1', '紧凑选项2'],
        dense=True,
        label='紧凑模式',
        classes='rounded-full' # 圆角样式
    )

    # 3. 下拉菜单自定义高度和宽度
    ui.select(
        options=[f'选项{i}' for i in range(10)],
        label='自定义下拉菜单',
    )

```

```

        props='menu-max-height=200 menu-width=300', # menu-max-height: 下拉菜单最大高度; menu-
width: 下拉菜单宽度
        hint='下拉菜单最大高度200px, 宽度300px'
    )

# 4. 禁用特定选项 (通过 Quasar 选项属性)
ui.select(
    options=[
        {'label': '可选择选项', 'value': 'enable'},
        {'label': '禁用选项', 'value': 'disable', 'disable': True} # 禁用该选项
    ],
    label='包含禁用选项',
    on_change=lambda e: ui.notify(f'选中: {e.value}')
)

ui.run()

```

效果:

- 四个 select 分别展示不同样式, 支持颜色、尺寸、圆角、下拉菜单大小的定制;
- 最后一个 select 中, “禁用选项” 不可点击选择, 仅作为展示。

5. 动态修改选项与选中值

通过 `set_options` 和 `set_value` 方法, 动态更新选项列表和选中值, 适用于动态场景:

```

from nicegui import ui

# 初始选项
select = ui.select(
    options=['初始选项1', '初始选项2'],
    label='动态修改示例',
    on_change=lambda e: ui.notify(f'选中: {e.value}')
)

# 动态添加选项
def add_option():
    new_options = select.options + [f'新增选项{len(select.options) + 1}']
    select.set_options(new_options)
    select.update() # 刷新界面

# 动态设置选中值
def set_default():
    select.set_value('初始选项1')

# 控制按钮
with ui.row().classes('mt-2 gap-2'):
    ui.button('添加选项', on_click=add_option)
    ui.button('重置选中值', on_click=set_default)

ui.run()

```

效果：点击“添加选项”按钮，选项列表新增一个选项；点击“重置选中值”按钮，选中值自动切换为“初始选项 1”。

四、核心属性与方法

1. 常用属性

属性名	类型	说明
classes	Classes[Self]	CSS 类（支持 Tailwind/Quasar 样式）
disabled	BindableProperty	是否禁用（可绑定数据动态控制）
html_id	str	HTML 元素 ID（2.16.0+ 版本支持，用于 teleport 定位）
label	str	组件标签（显示在选择框上方）
hint	str	提示文本（显示在选择框下方）
multiple	bool	是否启用多选模式
options	list / dict / Callable	选项配置（可动态修改）
placeholder	str	未选中时的提示文本
searchable	bool	是否支持搜索功能
clearable	bool	是否支持清空选中值
value	BindableProperty	当前选中值（单选为单个值，多选为列表，可绑定数据）
visible	BindableProperty	是否可见（可绑定数据）
props	Props[Self]	Quasar 特性属性（如 color、size、menu-max-height 等）

2. 关键方法

(1) 状态控制

- `disable()`：禁用组件（不可交互，视觉灰度）
- `enable()`：启用组件
- `set_disabled(value: bool)`：设置禁用状态（True/False）
- `set_visibility(visible: bool)`：设置组件可见性

(2) 选项与值修改

- `set_options(options: list / dict / Callable, value: Any = Ellipsis)`：动态更新选项，可选参数 `value` 用于设置新选中值（未指定则保留当前值）
- `set_value(value: Any / list)`：动态设置选中值（单选为单个值，多选为列表，触发 `on_change` 事件）
- `clear()`：清空选中值（等价于 `set_value(None)` 或 `set_value([])`（多选））
- `update()`：修改选项或属性后，调用此方法刷新界面

(3) 事件与绑定

- `on_change(callback)`: 绑定选中值变化事件 (`e.value` 为当前选中值)
- `on(type: str, handler)`: 订阅任意 DOM 事件 (如 `click`、`input`)
- `bind_value(target_object, target_name)`: 双向绑定选中值到目标对象的属性
- `bind_disabled_from(target_object, target_name)`: 单向绑定禁用状态从目标对象

(4) 其他实用方法

- `tooltip(text: str)`: 为组件添加悬停提示
- `delete()`: 彻底删除组件
- `mark(*markers)`: 添加标记 (用于测试或元素查询)
- `focus()`: 让组件获取焦点 (激活输入状态)

五、使用场景与注意事项

1. 适用场景

- 表单录入: 如用户注册时的性别、学历、所在城市等选择项。
- 筛选条件: 如列表数据的分类筛选 (如按状态、类型、时间范围筛选)。
- 配置选择: 如系统设置中的主题、语言、权限等配置项。
- 多选项选择: 如标签选择、角色分配等需要选择多个选项的场景。

2. 关键注意事项

- 选项格式一致性: 列表格式中选项值即标签, 字典格式中键为值、值为标签, 多选模式下 `value` 必须为列表 (即使仅选中一个选项)。
- 动态选项加载: 若 `options` 为可调用对象, 组件初始化时会自动调用一次加载选项, 若需动态刷新选项, 需手动调用 `set_options` 并传入新的可调用对象或列表 / 字典。
- 多选模式交互: 多选时需按住 `Ctrl` 键点击选项 (Windows/Linux) 或 `Command` 键 (Mac), 若需取消选中, 再次点击该选项即可。
- 搜索功能限制: 搜索仅匹配选项的标签 (列表格式匹配值, 字典格式匹配标签), 不支持模糊匹配或自定义搜索逻辑 (需通过自定义组件实现)。
- 版本兼容性: `html_id` 属性仅在 2.16.0+ 版本支持, `bind_disabled` 的 `strict` 参数在 3.0.0+ 版本支持, 使用时需确认版本。
- 与 `ui.radio` 的区别: `ui.select` 适用于选项较多的场景 (下拉菜单节省空间), 支持搜索和多选; `ui.radio` 适用于选项较少的场景 (直观展示所有选项), 仅支持单选。

六、进阶示例: 带搜索和分页的动态选项 select

结合 `ui.input` 和分页逻辑, 实现支持搜索和分页的动态选项 `select`, 适用于选项数量庞大的场景:

```
from nicegui import ui

# 模拟大量选项数据 (100个选项)
```



```

all_options = [f'选项{i}' for i in range(1, 101)]
page_size = 10 # 每页显示10个选项
current_page = 1
current_search = ''

# 初始化 select 组件（选项后续动态更新）
select = ui.select(
    options=[],
    label='带搜索和分页的选择框',
    placeholder='搜索选项...',
    on_change=lambda e: ui.notify(f'选中: {e.value}')
)

# 搜索输入框
search_input = ui.input(
    placeholder='输入关键词搜索...',
    on_change=lambda e: update_options(e.value, 1)
).classes('mt-2')

# 分页控制按钮
page_controls = ui.row().classes('mt-2 gap-2')
prev_btn = ui.button('上一页', on_click=lambda: update_options(current_search, current_page - 1), disabled=True)
page_label = ui.label(f'第 {current_page} 页')
next_btn = ui.button('下一页', on_click=lambda: update_options(current_search, current_page + 1))

# 更新选项和分页状态的函数
def update_options(search: str, page: int):
    global current_page, current_search
    current_search = search
    current_page = page

    # 过滤选项（匹配搜索关键词）
    filtered = [opt for opt in all_options if search.lower() in opt.lower()]
    total_pages = (len(filtered) + page_size - 1) // page_size # 总页数

    # 分页截取选项
    start = (page - 1) * page_size
    end = start + page_size
    paginated_options = filtered[start:end]

    # 更新 select 选项
    select.set_options(paginated_options)
    select.update()

    # 更新分页按钮状态和标签
    prev_btn.set_disabled(page <= 1)
    next_btn.set_disabled(page >= total_pages)
    page_label.set_text(f'第 {page} 页 / 共 {total_pages} 页')

# 初始加载第一页选项
update_options('', 1)

```

```
ui.run()
```

效果：

- 支持输入关键词搜索选项，实时过滤匹配结果；
- 下拉菜单每页显示 10 个选项，通过“上一页”“下一页”按钮切换分页；
- 分页按钮状态自动适配（第一页禁用上一页，最后一页禁用下一页），页面标签实时显示当前页数和总页数。